

G03 Gate A - Retail Fulfillment Reliability

AVAILABLE only after M04+M05+M06 are complete and evidenced.

Assessment rule: close teaching books, videos, protected Reviews and AI solving. You may use normal language/runtime help only after the attempt is frozen, as part of repair. Save evidence before Review.

Scenario

You inherit a small Retail Fulfillment Reliability data drop plus paginated JSON activity. Build a safe command-line pipeline and a reviewable repository that another engineer can reproduce.

Part 1 - plan before code (K1/K7)

1. State the grain of orders.csv and customers.csv. Identify candidate keys and the relationship. 2. Write the accepted-vs-quarantine contract before coding: duplicate IDs, bad dates, invalid/non-positive amount, unmatched customer_id, and non-unique dimension keys must stay visible. 3. Sketch functions/modules so parsing, validation, transformation and CLI/filesystem effects are separable.

Part 2 - Python/files/pandas (K1/K2/K4)

4. Preserve raw inputs. Parse CSV deliberately and normalize types without silently coercing bad records. 5. Produce accepted and quarantine outputs with reason codes. 6. Join accepted facts to the entity table only after proving the dimension key is unique; prevent fan-out. 7. For accepted status=Completed, aggregate count and amount by region. Save independent source-to-output controls. 8. Include at least one clean function, one explicit exception boundary, logging and automated tests for a valid case plus two failure cases.

Part 3 - paginated input + resilience (K3/K4)

9. Read both bundled API JSON pages as if they arrived sequentially. Follow the payload next/cursor field rather than hard-coding a single page. Detect duplicate event/case IDs instead of silently double-counting them. 10. Write a requests-style boundary that includes timeout, status checking and bounded retry/rate-limit reasoning, but keep the Gate deterministic by testing against the bundled JSON fixtures. 11. Configuration comes from CLI/environment/default precedence. Commit only .env.example; use fake secrets. 12. Add a DB-write interface or SQLAlchemy-shaped boundary that can be tested without requiring a live database. If you have the course PostgreSQL runtime available, prove one safe connection/transaction separately; otherwise mark it for final runtime QA - do not fake it.

Part 4 - Linux/WSL + shell (K5)

13. Work in a safe Linux workspace. Use inspection commands/pipes/redirection to produce one evidence file. Make the runner executable with least privilege. Demonstrate meaningful exit codes for success and at least one input failure. Create/recreate .venv and prove it is ignored. Do not use chmod 777, blind sudo or destructive recursive deletion as a repair shortcut.

Part 5 - Git/GitHub engineering workflow (K6)

14. Initialize Git, create focused commits and show committed/staged/unstaged state deliberately. 15. Create two branches that make these incompatible runbook statements: "Retry failed batches after validating the source window." and "Never retry a failed batch until the failure class is recorded.". Merge, resolve the conflict by preserving the required operational meaning, and prove clean status. 16. Demonstrate restore on an uncommitted mistake and revert on a committed mistake; save reflog evidence for one reference movement. 17. Use a local bare remote + second clone to prove push/fetch/pull semantics. If GitHub is available, open/review/merge a PR; otherwise save a PR-style diff/summary packet for the later learner-account smoke.

Part 6 - clean-clone handoff + timed defense (K7)

18. From a clean clone, recreate the environment from committed instructions, rerun validation and prove controls match. 19. Save README/setup, commands, tests, logs, accepted/quarantine summaries, Git graph/status and clean-clone evidence. 20. In 10-15 minutes, without Review open, explain the input grain, biggest quality risk, join-fanout defense, pagination/duplicate defense, retry boundary, secret/config boundary, shell exit-code contract, Git conflict/recovery choices, and what proves reproducibility.

Gate decision record

Competency	Status	Evidence reference / critical defect
K1 - Python reasoning and core language control	DEMONSTRATED / REPAIR_REQUIRED	
K2 - Files, pandas, grain, joins and validation	DEMONSTRATED / REPAIR_REQUIRED	
K3 - APIs, pagination, retries, configuration and secret boundaries	DEMONSTRATED / REPAIR_REQUIRED	
K4 - Reusable pipeline structure, logging, testing and DB interface discipline	DEMONSTRATED / REPAIR_REQUIRED	
K5 - Linux/WSL shell, permissions, venv and command safety	DEMONSTRATED / REPAIR_REQUIRED	
K6 - Git/GitHub state, branches, conflicts, remotes, PRs and recovery	DEMONSTRATED / REPAIR_REQUIRED	
K7 - Integrated reproducibility, evidence and technical explanation	DEMONSTRATED / REPAIR_REQUIRED	

Reviewer route

If K1-K7 are all demonstrated and no critical defect remains: GATE_FAMILY_PASS -> M07. Otherwise preserve the attempt, open only the matching protected Review, assign targeted changed repair, and keep the next full Gate sealed unless explicitly required.